

Git & GitHub for DevOps Complete Curriculum

Approach 3: Story-Mode ("DevOps Team Onboarding")

Beginner-friendly | 5 Sessions | 1 Hour Each | Theory + Practical Interwoven

Instructor Guide

Philosophy

This curriculum treats learning Git like joining a real DevOps team. Students are not memorizing abstract commands — they are living a story. Every session is a "day at work." This creates emotional context, which is how humans remember things best.

Magic Rule: If a 5-year-old can't understand the analogy, don't use it.

Pacing:

- Each session is exactly 1 hour
- 15 min: Story setup + theory (slides)
- 30 min: Hands-on lab
- 10 min: Practice exercise (student does it alone)
- 5 min: Review + Q&A

Student prerequisites: None. Absolutely none. They don't need to know what a terminal is.

Required environment:

- Computer with internet
- Terminal access (Linux/Mac native, Git Bash on Windows)
- Git installed (see Environment Setup)

- GitHub account (free)
- Text editor (VS Code recommended, but Notepad works for Session 1)

Before You Start

Make students do the "Environment Setup" section on their own BEFORE the first session. It takes 20-30 minutes. Do NOT use class time for this unless they are on locked-down machines.

Environment Setup Guide

Do this before Day 1. Every command below includes the environment where it runs.

Check Your Environment

First, figure out what computer you have. It's like knowing whether you drive a car, truck, or bicycle.

If you see...	You have
A Windows logo or <code>C:\</code> drive	Windows
A Finder or <code>~/</code> path	Mac
A Penguin or <code>apt-get</code>	Linux

Step 1: Install Git

Environment: Windows

1. Go to `https://git-scm.com/download/win`
2. Download the installer
3. Run it. When it asks questions, just click **Next** on everything (defaults are fine for beginners)
4. Open **Git Bash** (search for it in Start menu)
5. Verify: type the command below and press Enter

```
$ git --version
```

Expected Output:

```
git version 2.43.0.windows.1
```

(Your number might be different — that's fine!)

Environment: Mac

1. Open **Terminal** (press Cmd+Space, type `terminal`, press Enter)
2. Type this command:

```
$ git --version
```

Expected Output:

```
git version 2.39.3
```

If it says "command not found" or asks to install "Developer Tools", click **Install** and wait 5 minutes. Then try again.

Environment: Linux (Ubuntu/Debian)

1. Open your terminal (Ctrl+Alt+T usually)
2. Type:

```
$ sudo apt update && sudo apt install git -y
```

1. Verify:

```
$ git --version
```

Expected Output:

```
git version 2.34.1
```

Environment: Linux (Fedora/CentOS/RHEL)

```
$ sudo dnf install git -y  
$ git --version
```

Step 2: Configure Git (Tell Git Who You Are)

Analogy: *Git is like a diary. It needs to know your name so it can write "By [Your Name]" on every page.*

Run these commands in your terminal (Git Bash on Windows, Terminal on Mac/Linux).

Environment: Any terminal (Git Bash / Terminal)

```
$ git config --global user.name "Your Full Name"  
$ git config --global user.email "youremail@example.com"
```

Use the SAME email you will use for GitHub.

Verify it worked:

```
$ git config --global user.name
```

Expected Output:

```
Your Full Name
```

```
$ git config --global user.email
```

Expected Output:

```
youremail@example.com
```

Step 3: Create a GitHub Account

Analogy: *GitHub is like a library where you store your diary so others can read it.*

1. Go to `https://github.com` in your browser
2. Click **Sign up**
3. Use the SAME email you used in Step 2
4. Create a password and username
5. Click **Create account**
6. Check your email for a verification code and enter it

Done! You now have a library card.

Step 4: Set Up SSH Keys (The Secret Handshake)

Analogy: *SSH is like a secret handshake. It proves to GitHub that your computer really belongs to you, so you don't need to type your password every time.*

Environment: Any terminal (Git Bash / Terminal)

```
$ ssh-keygen -t ed25519 -C "youremail@example.com"
```

You will see:

```
Generating public/private ed25519 key pair.  
Enter file in which to save the key (/home/you/.ssh/id_ed25519):
```

Just press **Enter** (accept the default).

Next:

```
Enter passphrase (empty for no passphrase):
```

Press **Enter** again (twice) to skip a password. This is fine for learning.

Expected final output:

```
Your identification has been saved in /home/you/.ssh/id_ed25519
Your public key has been saved in /home/you/.ssh/id_ed25519.pub
```

Now copy your public key to the clipboard:

Mac:

```
$ pbcopy < ~/.ssh/id_ed25519.pub
```

Windows (Git Bash):

```
$ cat ~/.ssh/id_ed25519.pub | clip
```

Linux:

```
$ cat ~/.ssh/id_ed25519.pub
```

Select the output and copy it manually.

Now add it to GitHub:

1. Go to `https://github.com` in your browser
2. Click your profile picture (top right) → **Settings**
3. In the left sidebar, click **SSH and GPG keys**
4. Click **New SSH key**
5. Title: type "My Laptop"
6. Key: paste what you copied (it starts with `ssh-ed25519`)
7. Click **Add SSH key**

Test it:

```
$ ssh -T git@github.com
```

Expected Output:

```
The authenticity of host 'github.com (...)' can't be established...
Are you sure you want to continue connecting? (yes/no/[fingerprint])?
```

Type `yes` and press Enter.

Final expected output:

```
Hi yourusername! You've successfully authenticated, but GitHub does not provide shell
```

📦 Environment setup complete!

Step 5: Install a Text Editor

Recommended: VS Code (<https://code.visualstudio.com>)

- Install it from the website
- For beginners, it's friendly and free

Alternative: You can use Notepad (Windows), TextEdit (Mac), or Nano/Gedit (Linux) for this course.

Session 1: "Day 1 at the Company"

Learning the Tools + Your First Repository

Learning Objectives

By the end of this session, students will:

- Understand what version control is, using a simple analogy
- Create their first Git repository
- Make their first commit
- Read Git status like a dashboard

Slide Outline (15 minutes)

Slide 1: Title — "Welcome to TechCorp! You're the new DevOps engineer."

Slide 2: The Problem — "Have you ever saved a file as `report_final.doc` , then `report_final2.doc` , then `report_final_ACTUAL.doc` ?"

Slide 3: The Analogy — "Git is like a time machine for your files. Every time you save a version, Git can take you back to that exact moment."

Slide 4: DevOps Context — "In a company, 10 people edit the same files. Without Git, they'd overwrite each other. With Git, everyone has their own copy and Git merges the changes."

Slide 5: Key Terms (with icons)

- **Repository (Repo):** A project folder that Git watches
- **Commit:** A snapshot of your files at one moment in time
- **Working Directory:** Your normal files (like a messy desk)
- **Staging Area:** A box where you put files before taking a photo of them
- **History:** All the snapshots/commits you've ever made

Slide 6: Today's Mission — "Create your first repo and save your first snapshot."

Theory Script (Read or Present)

"Imagine you're drawing a picture. You draw a sun, then a house, then a tree. Now your little sister draws a monster on it. You cry because your beautiful picture is ruined!"

What if you had a magic camera? Every time you finish something good, you take a photo. If your sister ruins it, you just say 'take me back to photo #2!' and POOF — your picture is safe again.

Git is that magic camera. Every photo is called a **commit**. The folder where you keep all your drawings AND the magic camera is called a **repository** (or repo for short).

Today, you're joining a pretend company called TechCorp. Your boss says: 'Please set up our project folder.' You will create a repository for a simple website."

Practical Lab — "Create Your First Repo" (30 minutes)

Story: "TechCorp's website is just a text file right now. Your job is to put it under version control so your team can track changes."

Step 1: Create a Folder

Environment: Terminal (Git Bash / Terminal)


```
$ cd ~  
$ mkdir techcorp-website  
$ cd techcorp-website
```

Expected Output:

(No output for `mkdir` and `cd` — that's normal!)

Verify where you are:

```
$ pwd
```

Expected Output:

```
/home/you/techcorp-website
```

(Mac/Windows path will look different, but should end with `techcorp-website`)

Step 2: Check Git Status

```
$ git status
```

Expected Output:

```
fatal: not a git repository (or any of the parent directories): .git
```

Explain: "Git is not watching this folder yet. It's like a security guard who hasn't been hired for this building."

Step 3: Initialize the Repository (Hire the Security Guard)

```
$ git init
```

Expected Output:

```
Initialized empty Git repository in /home/you/techcorp-website/.git/
```

Explain: "The `.git` folder is Git's office. It stores every photo/commit. Never delete it!"

Now check status again:

```
$ git status
```

Expected Output:

```
On branch main
No commits yet
nothing to commit (create/copy files and use "git add" to track)
```

Step 4: Create a File

Environment: Terminal (or use a text editor if you prefer)

```
$ echo "Welcome to TechCorp!" > index.txt
```

Explain: "The `echo` command writes text into a file. The `>` symbol means 'put this into that file.'"

Check what's inside:

```
$ cat index.txt
```

Expected Output:

```
Welcome to TechCorp!
```

Step 5: Check Git Status Again

```
$ git status
```

Expected Output:

```
On branch main
No commits yet
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  index.txt
```

Explain: "Git sees the file! But it says 'untracked.' That means Git knows the file exists but hasn't taken a photo of it yet. Imagine the security guard saying 'I see a painting on the floor, but I haven't recorded it in my logbook.'"

Step 6: Add the File to the Staging Area

Analogy: *The staging area is like a box where you put things before you take a photo. You might put a toy in the box, then decide you don't want it in the photo, so you take it out. That's staging.*

```
$ git add index.txt
```

Expected Output:

(No output — silence means success in Git!)

Check status:

```
$ git status
```

Expected Output:

```
On branch main
No commits yet
Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   index.txt
```

Explain: "Git now says 'new file: index.txt' under 'Changes to be committed.' That means the file is IN the staging box, ready for the photo."

Step 7: Take the Photo (Make Your First Commit)

```
$ git commit -m "Add homepage with welcome message"
```

Expected Output:

```
[main (root-commit) abc1234] Add homepage with welcome message
1 file changed, 1 insertion(+)
create mode 100644 index.txt
```

Explain what each part means:

- `main` = the default branch name (like the main storyline in a video game)
- `root-commit` = the very first commit (the beginning of history)
- `abc1234` = the commit's unique ID (like a photo number)
- The message in quotes = a note you write on the back of the photo so you remember what changed

Step 8: Check Your History

```
$ git log
```

Expected Output:

```
commit abc1234def5678 (HEAD -> main)
Author: Your Name <youremail@example.com>
Date:   [today's date]
```

```
    Add homepage with welcome message
```

Explain: "This is your time machine! `git log` shows every photo you've ever taken. HEAD → main means 'you are looking at the latest commit on the main branch.'"

Step 9: Change the File and See Git Track It

Let's edit the file to add a new line.

```
$ echo "We build amazing things." >> index.txt
```

Explain: `>>` means "add this to the end of the file." (Two arrows = append. One arrow = overwrite.)

Check the file:

```
$ cat index.txt
```

Expected Output:

```
Welcome to TechCorp!  
We build amazing things.
```

Check Git status:

```
$ git status
```

Expected Output:

```
On branch main  
Changes not staged for commit:  
  (use "git add <file>..." to update what will be committed)  
  (use "git restore <file>..." to discard changes in working directory)  
        modified:   index.txt
```

Explain: "Git sees the file changed! It says 'modified.' But it won't take a photo until we put it back in the staging box."

Stage and commit:

```
$ git add index.txt  
$ git commit -m "Add company description"
```

Expected Output:

```
[main def5678] Add company description  
1 file changed, 1 insertion(+)
```

Check log again:

```
$ git log
```

Expected Output:

```
commit def5678... (HEAD -> main)
Author: Your Name ...
Date: ...
```

Add company description

```
commit abc1234...
Author: Your Name ...
Date: ...
```

Add homepage with welcome message

📌 **Session 1 Complete!** You have a time machine with 2 saved states.

Student Practice Exercise (10 minutes)

Scenario: Create a file called `about.txt` with the text "Founded in 2024." Stage it and commit it with the message "Add about page."

📌 Solution

```
$ echo "Founded in 2024." > about.txt
$ git add about.txt
$ git commit -m "Add about page"
```

****Expected Output:****

```
[main ghi9012] Add about page
1 file changed, 1 insertion(+)
create mode 100644 about.txt
```

Verify:

```
$ git log --oneline
```

****Expected Output:****

```
ghi9012 Add about page
def5678 Add company description
abc1234 Add homepage with welcome message
```

Common Beginner Mistakes

Mistake	Why It Happens	Fix
<code>fatal: not a git repository</code>	Ran <code>git</code> commands outside of the repo	Use <code>cd</code> to navigate into the folder first
Commits but <code>git status</code> still shows changes	Forgot <code>git add</code> before <code>git commit</code>	Stage first, then commit
Typo in commit message	Rushed	It's okay! We'll learn to fix it in Session 2
Can't find <code>.git</code> folder	It's hidden by default	Use <code>ls -la</code> (Mac/Linux) or <code>dir /a</code> (Windows)

Review (5 minutes)

Ask the class:

1. "What is a repository?" (A folder Git watches)
 2. "What is a commit?" (A snapshot/photo of your files)
 3. "What's the staging area?" (A box where we prepare files before committing)
 4. "What do `git add` and `git commit` do together?" (Add = put in box. Commit = take photo)
-
-

Session 2: "Fixing Your First Bug"

Branching + Working on Features Without Breaking Main

Learning Objectives

By the end of this session, students will:

- Understand what a branch is (with a simple analogy)
- Create, switch between, and merge branches
- See how branches prevent breaking the main project

Slide Outline (15 minutes)

Slide 1: Title — "Oh No! The Website is Broken on Main!"

Slide 2: The Problem — "You need to fix a typo, but you don't want to break the live website while you're figuring out the fix."

Slide 3: The Analogy — "Imagine you're reading a choose-your-own-adventure book. The main story is one path. But you can bookmark a page, try a different path, and if it works, go back and rewrite the main story. A branch is a bookmarked alternate path."

Slide 4: DevOps Context — "In real teams, you NEVER edit the `main` branch directly. You create a feature branch, test it, then merge it back. This protects the live code."

Slide 5: Key Terms

- **Branch:** A parallel timeline (like a copy of the story)
- **git branch :** Creates a new timeline
- **git checkout (or git switch):** Jumps to a different timeline
- **Merge:** Combines two timelines back together

Slide 6: Today's Mission — "The boss says the homepage has a typo. Create a branch, fix it, and merge it back."

Theory Script

"In our story, our website is 'live' on the `main` branch. Imagine `main` is the highway that real customers drive on. If you start digging up the highway to fix a pothole, cars will crash!

So what do construction workers do? They close off one lane, work there, and only merge it back into the highway when it's safe.

*In Git, a **branch** is that closed-off lane. You create a branch, do your work safely, test it, and then **merge** it back into `main` .*

The magic is: your `main` branch stays safe the whole time. If you mess up on your branch, just delete it and start over. No customers were harmed."

Practical Lab — "The Typo Fix" (30 minutes)

Story: "A customer emailed: 'Your website says "We build amaizing things" — that's not a word!' Your task: Fix the typo without touching the live site."

Preparation

Make sure you're in your repo from Session 1.

Environment: Terminal (in `~/techcorp-website`)

```
$ cd ~/techcorp-website
```

Step 1: Check What Branch You're On

```
$ git branch
```

Expected Output:

```
* main
```

Explain: "The `*` means 'you are here.' Like a 'You Are Here' dot on a mall map."

Step 2: Create a New Branch

```
$ git branch fix-typo
```

Check branches again:

```
$ git branch
```

Expected Output:

```
fix-typo  
* main
```

Explain: "Notice: we created the lane, but we're still on the highway (`main`). The `*` is still on `main` ."

Step 3: Switch to the New Branch

```
$ git switch fix-typo
```

Expected Output:

```
Switched to branch 'fix-typo'
```

Verify:

```
$ git branch
```

Expected Output:

```
* fix-typo  
main
```

Explain: "The star moved! We're now working on the closed-off lane. Any changes we make here do NOT affect `main` yet."

Step 4: Make the Fix

Introduce a typo first (so we have something to fix):

```
$ echo "We build amaizing things." > index.txt
```

Wait — we already had good text. Let's see the real scenario. First, let's see what we have:

```
$ cat index.txt
```

Let's say it really does have the typo. Fix it:

```
$ echo "Welcome to TechCorp!" > index.txt  
$ echo "We build amazing things." >> index.txt
```

Better way — use a text editor:

Open `index.txt` in your text editor and change "amaizing" to "amazing", then save.

Check what Git sees:

```
$ git status
```

Expected Output:

```
On branch fix-typo
Changes not staged for commit:
  modified:   index.txt
```

Stage and commit:

```
$ git add index.txt
$ git commit -m "Fix typo: amaizing -> amazing"
```

Expected Output:

```
[fix-typo 123def0] Fix typo: amaizing -> amazing
1 file changed, 1 insertion(+), 1 deletion(-)
```

Step 5: Check That Main is Still Safe

```
$ git switch main
```

Look at the file:

```
$ cat index.txt
```

Expected Output: (The OLD version — with the typo still there!)

```
Welcome to TechCorp!
We build amaizing things.
```

Explain: "HALLELUJAH! `main` is safe. Our fix is ONLY on the `fix-typo` branch. The highway is still open."

Switch back:

```
$ git switch fix-typo
$ cat index.txt
```

Expected Output: (The fixed version)

```
Welcome to TechCorp!
We build amazing things.
```

Step 6: Merge the Fix Back Into Main

Now that we tested the fix, we merge the lane back into the highway.

```
$ git switch main
$ git merge fix-typo
```

Expected Output:

```
Updating def5678..123def0
Fast-forward
 index.txt | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Explain: "Fast-forward means Git just moved the `main` pointer forward. No complicated merging needed — it was a straight line."

Verify:

```
$ cat index.txt
```

Expected Output:

```
Welcome to TechCorp!
We build amazing things.
```

Step 7: Delete the Branch (Cleanup)

The lane has been merged into the highway. We don't need the lane anymore.

```
$ git branch -d fix-typo
```

Expected Output:

```
Deleted branch fix-typo (was 123def0).
```

Verify:

```
$ git branch
```

Expected Output:

```
* main
```

Check the log:

```
$ git log --oneline
```

Expected Output:

```
123def0 Fix typo: amaizing -> amazing  
def5678 Add company description  
abc1234 Add homepage with welcome message
```

☐ **Bug fixed, main is safe, history is clean!**

Student Practice Exercise (10 minutes)

Scenario: The boss wants a "Contact Us" page. Create a branch called `add-contact`. Add a file `contact.txt` with the text `Email: hello@techcorp.com`. Commit it. Switch back to `main` and verify `contact.txt` does NOT exist there. Then merge `add-contact` into `main` and verify it DOES exist. Finally, delete the branch.

☐ **Solution**

```
# Create and switch to branch
$ git branch add-contact
$ git switch add-contact

# Create file and commit
$ echo "Email: hello@techcorp.com" > contact.txt
$ git add contact.txt
$ git commit -m "Add contact page"

# Verify main doesn't have it
$ git switch main
$ ls
```

****Expected Output:**** `about.txt index.txt` (note: NO `contact.txt`)

```
# Merge the branch
$ git merge add-contact
$ ls
```

****Expected Output:**** `about.txt contact.txt index.txt`

```
# Cleanup
$ git branch -d add-contact
```

Common Beginner Mistakes

Mistake	Why It Happens	Fix
Edited files on <code>main</code> directly	Forgot to create/switch branch	Create branch from the fixed commit, or learn <code>git stash</code> (Session 4)
<code>git merge</code> says "Already up to date"	Tried to merge in the wrong direction	Make sure you're ON <code>main</code> and merging the feature branch INTO it
Can't delete branch	Used lowercase <code>-d</code> when branch isn't merged	Use <code>-D</code> (uppercase) to force delete, or merge first

Session 3: "Submitting Your Work"

GitHub Remotes, Push, Pull, and Pull Requests

Learning Objectives

By the end of this session, students will:

- Understand what a "remote" is (with analogy)
- Push their local repo to GitHub
- Create and merge a Pull Request
- Understand the code review workflow

Slide Outline (15 minutes)

Slide 1: Title — "Time to Show the Team!"

Slide 2: The Problem — "Your code is on your laptop. Your boss can't see it. Your teammates can't see it. How do you share?"

Slide 3: The Analogy — "Your laptop is like your bedroom desk. GitHub is like the school library. You do your homework at your desk, then you bring it to the library so your teacher and classmates can see it too. Pushing is 'bringing it to the library.' Pulling is 'getting the latest homework from the library.'"

Slide 4: DevOps Context — "In real teams, you never push directly to `main` on GitHub. You push your branch, open a Pull Request (PR), ask someone to review it, THEN merge it. This is how bugs get caught before they reach customers."

Slide 5: Key Terms

- **Remote:** A copy of your repo on another computer (usually GitHub)
- **Push:** Send your commits to the remote
- **Pull:** Download commits from the remote
- **Clone:** Make a copy of a remote repo on your computer
- **Pull Request (PR):** Asking permission to merge your branch into main

Slide 6: Today's Mission — "Put your repo on GitHub. Create a feature branch. Open a PR. Get it approved (by yourself for practice) and merge it."

Theory Script

"So far, all your code is on your laptop. Imagine doing homework on your bedroom desk and never showing anyone. That's fine for practice, but in a real job, your TEAM needs to see your work."

GitHub is like a library. You push (upload) your code there. Your team can review it, comment on it, and suggest changes.

But here's the rule at TechCorp: Nobody is allowed to change the library's master copy directly. You must open a **Pull Request** — it's like filling out a form that says 'I think my homework is correct. Please review it before putting it in the official folder.'

Another engineer reviews your code, says 'Looks good!' and clicks merge. This is called **code review** and it's one of the most important practices in DevOps. It catches mistakes before they reach real customers."

Practical Lab — "First PR" (30 minutes)

Story: "The boss wants an FAQ page. You must add it, push it to GitHub, open a Pull Request, review it yourself, and merge it."

Step 1: Create a GitHub Repository

Environment: Browser (GitHub.com)

1. Go to `https://github.com` and log in
2. Click the + button (top right) → **New repository**
3. Repository name: `techcorp-website`
4. Description: "TechCorp official website"
5. Choose **Public** (Private costs money for teams)
6. **DO NOT** check "Initialize this repository with a README"
7. Click **Create repository**

You will see a page with instructions. Look for the section titled **"...or push an existing repository from the command line."**

Step 2: Connect Your Local Repo to GitHub

Environment: Terminal (in `~/techcorp-website`)

```
$ git remote add origin git@github.com:YOURUSERNAME/techcorp-website.git
```

Replace YOURUSERNAME with your actual GitHub username.

Verify it worked:

```
$ git remote -v
```

Expected Output:

```
origin  git@github.com:YOURUSERNAME/techcorp-website.git (fetch)
origin  git@github.com:YOURUSERNAME/techcorp-website.git (push)
```

Explain: " `origin` is just a nickname for the remote server. Like saving a phone number as 'Mom' instead of typing the full number every time."

Step 3: Push Main to GitHub

```
$ git push -u origin main
```

Expected Output:

```
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
... (more lines)
To github.com:YOURUSERNAME/techcorp-website.git
 * [new branch]      main -> main
```

Explain: "Your code is now on GitHub! `-u origin main` tells Git: 'Remember that from now on, when I say `git push` on the `main` branch, push to `origin`.'"

Refresh your GitHub page in the browser. You should see:

- `index.txt` file
- `about.txt` file
- Your commit history

Step 4: Create a Feature Branch for the FAQ Page

```
$ git branch add-faq  
$ git switch add-faq
```

Step 5: Create the FAQ File

```
$ echo "Q: What does TechCorp do?" > faq.txt  
$ echo "A: We build amazing DevOps tools." >> faq.txt
```

Check what you made:

```
$ cat faq.txt
```

Expected Output:

```
Q: What does TechCorp do?  
A: We build amazing DevOps tools.
```

Stage and commit:

```
$ git add faq.txt  
$ git commit -m "Add FAQ page"
```

Step 6: Push the Branch to GitHub

```
$ git push -u origin add-faq
```

Expected Output:

```
* [new branch]      add-faq -> add-faq
```

Refresh GitHub. You will now see a yellow banner: "**add-faq had recent pushes. Compare & pull request.**"

Step 7: Open a Pull Request

Environment: Browser (GitHub.com)

1. Click the yellow banner's **"Compare & pull request"** button
2. Title: `Add FAQ page`
3. Description:
 - Added FAQ page with common questions - No breaking changes
4. Click **Create pull request**

Step 8: Review and Approve the PR

Since you're practicing alone, you will review your own PR.

1. Scroll down to see the "Files changed" tab or click it
2. You should see `faq.txt` with a green **+** showing the new content
3. Click the **Review changes** button (top right of the file)
4. Select **Approve**
5. Click **Submit review**

Step 9: Merge the PR

1. Click the **Conversation** tab
2. Click the green **Merge pull request** button
3. Click **Confirm merge**
4. Click **Delete branch** (cleanup)

Step 10: Pull the Changes to Your Local Main

"Your local `main` doesn't know about the merge yet. You need to pull (download) it."

Environment: Terminal

```
$ git switch main  
$ git pull
```

Expected Output:

```
remote: Enumerating objects: ...
Unpacking objects: 100% ...
From github.com:YOURUSERNAME/techcorp-website
    123def0..789abcd  main      -> origin/main
Updating 123def0..789abcd
Fast-forward
  faq.txt | 2 ++
  1 file changed, 2 insertions(+)
```

Verify locally:

```
$ ls
```

Expected Output:

```
about.txt  faq.txt  index.txt
```

```
$ git log --oneline
```

Expected Output: (includes the merge)

```
789abcd Add FAQ page
... (older commits)
```

Delete the local branch:

```
$ git branch -d add-faq
```

🎉 **Your first Pull Request is complete!**

Student Practice Exercise (10 minutes)

Scenario: Add a `services.txt` file with the text `Web Hosting, CI/CD, Cloud Consulting`. Do it all through a PR workflow:

1. Create branch `add-services`
2. Create file, commit
3. Push branch, open PR, merge on GitHub
4. Pull to local main
5. Delete branch everywhere

▮ Solution

```
# Local work
$ git branch add-services
$ git switch add-services
$ echo "Web Hosting, CI/CD, Cloud Consulting" > services.txt
$ git add services.txt
$ git commit -m "Add services page"

# Push and PR
$ git push -u origin add-services
```

****Then on GitHub:**** - Click "Compare & pull request" - Create PR, review, merge, delete branch ****Back in terminal:****

```
$ git switch main
$ git pull
$ git branch -d add-services
```

Verify:

```
$ ls
$ cat services.txt
```

Common Beginner Mistakes

Mistake	Why It Happens	Fix
<code>Permission denied</code> on push	SSH not set up correctly	Redo the SSH setup in Environment Setup
Pushed to <code>main</code> directly	Forgot to create a branch	Talk about branch protection rules (Session 5)
PR shows way more changes than expected	Branched from wrong place	Always branch from the latest <code>main</code>
<code>git pull</code> shows merge conflicts	Someone else changed the same file	Session 4 covers this

Session 4: "The Deploy Broke!"

Undoing, Reverting, and Handling Conflicts

Learning Objectives

By the end of this session, students will:

- Use `git log` effectively to find old commits
- Revert a bad commit safely
- Resolve a simple merge conflict
- Understand `git stash` for emergency context switches

Slide Outline (15 minutes)

Slide 1: Title — "3 AM Page: Production is Down!"

Slide 2: The Problem — "You just deployed code, and the website shows a white screen. The boss is calling. You need to undo the change FAST."

Slide 3: The Analogy — "You have a time machine (`git log`). But you can't erase the past — people might have seen it. Instead, you add a NEW commit that undoes the bad one. It's like saying 'Forget my last move' in a board game."

Slide 4: DevOps Context — "In DevOps, you rarely 'delete' history because audit trails matter. Instead, you **revert** — create a new commit that undoes an old one. This is safe, traceable, and professional."

Slide 5: Key Terms

- `git log --oneline` : A compact history view
- `git revert` : Create a new commit that undoes an old commit
- **Merge Conflict**: Two people changed the same line; Git needs a human to decide which version to keep
- `git stash` : Put your current work in a backpack so you can do something else

Slide 6: Today's Mission — "Accidentally break something, find the bad commit, revert it, and handle a merge conflict."

Theory Script

"At 3 AM, your phone rings. The website is broken. The last deploy was 20 minutes ago. You know it worked before that."

Here's what you DON'T do: panic and delete files. Here's what you DO:

- 1. Look at the history (`git log --oneline`)*
- 2. Find the bad commit*
- 3. Revert it (`git revert BAD_COMMIT`)*
- 4. Push the revert to GitHub*

Why revert instead of delete? Because in a real company, there might be laws (compliance) saying you can't erase history. Also, other developers might have already built on top of your bad commit. Reverting creates a NEW commit that undoes the bad one — history stays intact, but the code goes back to working.

*Also today: **merge conflicts**. Imagine you and your teammate both edited the same sentence in a document. Git sees both versions and says 'I don't know which one you want.' YOU have to tell it. We'll practice that."*

Practical Lab Part 1 — "The Revert" (15 minutes)

Story: "Someone (you!) accidentally deleted the 'Contact Us' info and committed it. Revert that commit to restore it."

Step 1: Break Something on Purpose

Environment: Terminal (in repo, on `main`)

```
$ cd ~/techcorp-website  
$ git switch main
```

Delete a line from `index.txt` and overwrite with bad info:

```
$ echo "Oops, we deleted everything!" > index.txt  
$ git add index.txt  
$ git commit -m "Refactor homepage"
```

Check log:

```
$ git log --oneline
```

Expected Output: (something like)

```
bad1234 Refactor homepage
789abcd Add FAQ page
123def0 Fix typo: amaizing -> amazing
```

Step 2: Find the Bad Commit

```
$ git log --oneline
```

Expected Output:

```
bad1234 Refactor homepage
789abcd Add FAQ page
```

"The top one (`bad1234`) broke everything! We need to undo it."

Step 3: Revert the Bad Commit

```
$ git revert bad1234
```

An editor will open (Nano or Vim). It shows the default commit message:

```
Revert "Refactor homepage"

This reverts commit bad1234...
```

Save and exit:

- **Nano:** Press Ctrl+O, then Enter, then Ctrl+X
- **Vim:** Press Esc, type `:wq` , press Enter

Expected Output:


```
[main undo5678] Revert "Refactor homepage"  
1 file changed, 1 insertion(+), 1 deletion(-)
```

Verify:

```
$ cat index.txt
```

Expected Output: (The GOOD version is back!)

```
Welcome to TechCorp!  
We build amazing things.
```

Check the log:

```
$ git log --oneline
```

Expected Output:

```
undo5678 Revert "Refactor homepage"  
bad1234 Refactor homepage  
789abcd Add FAQ page
```

Explain: "Notice: the bad commit (`bad1234`) is STILL in history. We didn't erase it. We ADDED a new commit (`undo5678`) that undoes it. This is the professional way."

Push to GitHub:

```
$ git push
```

Practical Lab Part 2 — "The Merge Conflict" (15 minutes)

Story: "You and a teammate both tried to update the homepage headline. Git can't decide which version is right. You must choose."

Step 1: Simulate Two People Changing the Same File

Create a conflict manually. First, change `index.txt` on `main`:

```
$ echo "Welcome to SUPER TechCorp!" > index.txt
$ git add index.txt
$ git commit -m "Update headline on main"
```

Create a branch from BEFORE that commit:

```
$ git branch conflict-branch HEAD~1
$ git switch conflict-branch
```

Explain: HEAD~1 means "one commit before the latest." Like saying "the previous photo."

Now make a DIFFERENT change to the same file:

```
$ echo "Welcome to TechCorp Inc." > index.txt
$ git add index.txt
$ git commit -m "Update headline on branch"
```

Step 2: Try to Merge

```
$ git switch main
$ git merge conflict-branch
```

Expected Output:

```
Auto-merging index.txt
CONFLICT (content): Merge conflict in index.txt
Automatic merge failed; fix conflicts and then commit the result.
```

❏ **Conflict!**

Step 3: See What Git Did

```
$ cat index.txt
```

Expected Output:

```
<<<<<<< HEAD
Welcome to SUPER TechCorp!
=====
Welcome to TechCorp Inc.
>>>>>>> conflict-branch
```

Explain the markers:

- <<<<<<< HEAD = "This is what main has"
- ===== = "Separator"
- >>>>>>> conflict-branch = "This is what the branch has"

Step 4: Fix the Conflict

Edit `index.txt` to say what you want. Let's combine them nicely:

```
$ echo "Welcome to SUPER TechCorp Inc.!" > index.txt
```

Important: Remove the conflict markers! If you use `echo` to overwrite, they're automatically gone. If you use a text editor, manually delete `<<<<<<<`, `=====`, and `>>>>>>>` lines.

Step 5: Tell Git It's Fixed

```
$ git add index.txt
$ git commit -m "Merge branch 'conflict-branch' and resolve headline conflict"
```

Expected Output:

```
[main merge9012] Merge branch 'conflict-branch' and resolve headline conflict
```

Verify:

```
$ git log --oneline
$ cat index.txt
```

☐ **Conflict resolved professionally!**

Step 6: Git Stash (Bonus Tool)

```
$ git switch main
```

Make a quick change but don't commit:

```
$ echo "Under construction" >> index.txt
```

Check status:

```
$ git status
```

"Oh no! The boss calls and says fix something on `main` immediately, but you're not done with this change."

Stash it (put it in a backpack):

```
$ git stash
```

Expected Output:

```
Saved working directory and index state ...
```

"Your messy desk is now clean, but your work is saved in a backpack."

Do your urgent fix, then restore your work:

```
$ git stash pop
```

Expected Output:

```
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
   modified:   index.txt
```

"Your backpack is unpacked and your work is back on the desk!"

Student Practice Exercise (10 minutes)

Scenario:

1. Add an emoji to `about.txt` and commit it on `main`
2. Create branch `better-about` from BEFORE that commit
3. On `better-about`, change the SAME LINE to something different
4. Merge `better-about` into `main` — you'll get a conflict
5. Resolve it by keeping the best parts of both
6. Stash a pretend "work in progress" on `index.txt`, then pop it

▮ Solution

```
# Step 1
$ echo "Founded in 2024! 🦄" > about.txt
$ git add about.txt
$ git commit -m "Add emoji to about"

# Step 2 & 3
$ git branch better-about HEAD~1
$ git switch better-about
$ echo "Founded in 2024 – Excellence since day one." > about.txt
$ git add about.txt
$ git commit -m "Improve about page text"

# Step 4
$ git switch main
$ git merge better-about
# (conflict happens)

# Step 5
$ cat about.txt
# (see conflict markers)
$ echo "Founded in 2024! 🦄 Excellence since day one." > about.txt
$ git add about.txt
$ git commit -m "Resolve about page conflict"

# Step 6
$ echo "WIP: redesigning everything" >> index.txt
$ git stash
$ git stash pop
```

Common Beginner Mistakes

Mistake	Why It Happens	Fix
Edited conflict markers but didn't <code>git add</code>	Thought editing was enough	Must stage (<code>git add</code>) and commit after fixing
Left conflict markers in the file	Didn't clean up completely	Search for <code><<<<<<</code> in the file
<code>git revert</code> on the wrong commit	Didn't read <code>git log --oneline</code> carefully	Use <code>git show COMMIT_ID</code> to preview a commit
<code>git stash</code> and forgot where it went	Stashed too many things	<code>git stash list</code> shows all stashes

Session 5: "Automating Everything"

GitHub Actions: Continuous Integration for DevOps

Learning Objectives

By the end of this session, students will:

- Understand what CI/CD means with a simple analogy
- Create a GitHub Actions workflow file
- See a workflow run automatically on `push`
- Understand how Git events trigger automation

Slide Outline (15 minutes)

Slide 1: Title — "Robots Do the Boring Work"

Slide 2: The Problem — "Every time someone pushes code, the boss wants: 'Did it break anything? Does it build? Can we deploy it?' Manually checking this is exhausting."

Slide 3: The Analogy — "Imagine every time you put a letter in the mailbox, a robot automatically opens it, checks the spelling, rewrites it if needed,

photocopies it, and puts it on everyone's desk. That's CI/CD — robots doing checks every time you change code."

Slide 4: DevOps Context — "CI/CD is the HEART of DevOps. **Continuous Integration** = automatically test code when it's pushed. **Continuous Deployment** = automatically deploy if tests pass. Today we build a simple CI pipeline."

Slide 5: Key Terms

- **CI/CD:** Continuous Integration / Continuous Deployment
- **Workflow:** A recipe that tells GitHub what to do
- **Trigger:** An event that starts the workflow (like `push` , `pull_request`)
- **Runner:** A computer that GitHub rents to run your recipe
- **YAML:** The language used to write the recipe (it's like a shopping list)

Slide 6: Today's Mission — "Build a robot that automatically checks our `index.txt` exists every time we push code."

Theory Script

*"In a real DevOps job, your team pushes code 50 times a day. You can't have a human check every single one. That's where **robots** come in.*

***GitHub Actions** is a robot factory. You write a recipe in a special language called YAML. The recipe says: 'When someone pushes code, do these steps.'*

A typical recipe looks like:

- 1. Get a fresh computer (runner)*
- 2. Install the tools we need*
- 3. Check out our code*
- 4. Run tests (or, for our simple example, verify files exist)*
- 5. Report PASS or FAIL*

If the robot reports FAIL, the developer knows they broke something without anyone else having to tell them. This saves HOURS every day.

YAML looks scary but it's just indented lists. Like a recipe in a cookbook:

Recipe: Chocolate Cake Steps: - Preheat oven to 350 - Mix flour and

`sugar`

That's it. That's YAML."

Practical Lab — "Build Your First Robot" (30 minutes)

Story: "TechCorp wants a robot that automatically checks: 'Does our website have at least index.txt and about.txt?' Every time someone pushes code, the robot runs and reports green (pass) or red (fail)."

Step 1: Create the Workflow File

Environment: Terminal (in repo, on `main`)

```
$ cd ~/techcorp-website  
$ git switch main
```

GitHub Actions looks for workflow files in a special folder: `.github/workflows/`

Create it:

```
$ mkdir -p .github/workflows
```

Create the workflow file:


```
$ cat > .github/workflows/check-files.yml << 'EOF'
name: Check Website Files

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  check-files:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Check required files exist
        run: |
          if [ ! -f "index.txt" ]; then
            echo "ERROR: index.txt is missing!"
            exit 1
          fi
          if [ ! -f "about.txt" ]; then
            echo "ERROR: about.txt is missing!"
            exit 1
          fi
          echo "All required files present!"

EOF
```

Explain what each line does:

- `name:` = What the robot job is called (shows up in GitHub)
- `on:` = When does the robot wake up? (on `push` to `main` or on `pull_request`)
- `jobs:` = The list of tasks
- `runs-on: ubuntu-latest` = "Use a fresh Linux computer"
- `uses: actions/checkout@v4` = "Download our code onto that computer"
- `run: |` = "Run these shell commands"

Step 2: Commit and Push

```
$ git add .github/workflows/check-files.yml
$ git commit -m "Add CI workflow to check required files"
$ git push
```

Step 3: Watch the Robot Work

Environment: Browser (GitHub.com)

1. Go to your repo on GitHub
2. Click the **Actions** tab (next to Pull requests)
3. You should see your workflow running!
4. Click on the yellow dot → it turns green 🟢

If you click inside, you'll see each step:

- Checkout code → 🟢
- Check required files exist → 🟢 with "All required files present!"

Step 4: Break Something and See It Fail

Create a branch and intentionally delete a required file:

Environment: Terminal

```
$ git branch break-ci
$ git switch break-ci
$ git rm about.txt
$ git commit -m "Remove about page (this should fail CI)"
$ git push -u origin break-ci
```

Environment: Browser (GitHub.com)

1. Open the Pull Request for `break-ci`
2. GitHub will run the workflow automatically
3. It will show a red X 🛑 next to the commit
4. Inside Actions, you'll see:

```
ERROR: about.txt is missing!
```

Explain: "The robot caught the mistake BEFORE anyone merged it! This is the power of CI."

Don't merge this PR. Close it (click **Close pull request**) to keep your repo clean.

Step 5: View the Workflow File in Context

Environment: Terminal

Show what's in your `.github/workflows` folder:

```
$ ls .github/workflows/
```

Expected Output:

```
check-files.yml
```

Show all CI workflows in GitHub via command line:

```
$ gh workflow list
```

Expected Output:

NAME	STATE	ID
Check Website Files	active	12345678

(Requires `gh` CLI installed — optional for class, but cool to show.)

Step 6: Branch Protection (The Safety Lock)

Environment: Browser (GitHub.com)

1. Go to your repo → **Settings** tab
2. In the left sidebar, click **Branches**
3. Under "Branch protection rules," click **Add rule**
4. Branch name pattern: `main`
5. Check: **Require a pull request before merging**
6. Check: **Require status checks to pass before merging**
- Search for and select: `Check Website Files`
7. Click **Create**

Explain: "Now the `main` branch is LOCKED. Nobody can push directly to it. Every change must go through a PR, AND the robot must give a green checkmark. This is how real DevOps teams protect production."

Test it:

```
$ git switch main
$ echo "Direct push test" >> index.txt
$ git add index.txt
$ git commit -m "Try to push directly"
$ git push
```

Expected Output:

```
remote: error: GH006: Protected branch update failed...
remote: error: At least 1 approving review is required...
```

"BLOCKED! The safety lock works."

Revert that commit locally:

```
$ git reset --soft HEAD~1
$ git restore --staged index.txt
$ git checkout -- index.txt
```

(Don't worry about understanding these commands perfectly — just know they undo the last commit and restore the file.)

Student Practice Exercise (10 minutes)

Scenario: Add a second workflow (or extend the first one) that checks `services.txt` also exists. Commit it, push it, and verify it runs in the Actions tab.

▮ Solution

Edit ``.github/workflows/check-files.yml`` and add another check:

```
$ cat >> .github/workflows/check-files.yml << 'EOF'
- name: Check services file
  run: |
    if [ ! -f "services.txt" ]; then
      echo "ERROR: services.txt is missing!"
      exit 1
    fi
    echo "services.txt is present!"
EOF
```

Oops — that appends to the file but may mess up indentation. Better to use a text editor to add this block INSIDE the `steps:` section, keeping the same indentation. Then:

```
$ git add .github/workflows/check-files.yml
$ git commit -m "Extend CI to check services.txt"
$ git push
```

Watch it run in the Actions tab on GitHub.

Capstone Wrap-Up: Your DevOps Journey

Environment: Browser + Terminal

Have each student demonstrate:

1. Show their GitHub repo with all files
2. Show the commit history (`git log --oneline`)
3. Show a closed PR
4. Show a passing GitHub Action
5. Show branch protection on `main`

This is their portfolio piece: "I set up version control and CI for a project."

Common Beginner Mistakes

Mistake	Why It Happens	Fix
YAML indentation errors	Copied code without proper spaces	YAML uses 2 spaces (not tabs). Use an editor that shows spaces
Workflow doesn't appear	File in wrong folder	MUST be <code>.github/workflows/</code> (case sensitive)
Action fails immediately	Typo in <code>uses:</code> line	Double-check <code>actions/checkout@v4</code>
Branch protection too strict	Required checks not running yet	Make sure the workflow name matches exactly

Appendix A: Quick Reference Card

Essential Commands

Command	What It Does	When to Use
<code>git init</code>	Start Git in a folder	Beginning of a new project
<code>git status</code>	Check what's happening	ALL THE TIME. Before every commit.
<code>git add filename</code>	Put file in staging box	When a file is ready to be committed
<code>git commit -m "message"</code>	Take a snapshot	After staging
<code>git log --oneline</code>	See compact history	When you need to find an old commit
<code>git branch name</code>	Create a new branch	Starting a new feature
<code>git switch name</code>	Change branches	Moving between features
<code>git merge name</code>	Combine branches	Finishing a feature
<code>git push</code>	Upload to GitHub	Sharing your work
<code>git pull</code>	Download from GitHub	Getting the latest team code
<code>git revert COMMIT</code>	Undo a commit safely	Fixing a mistake in history
<code>git stash</code>	Save temporary work	Switching tasks urgently
<code>git stash pop</code>	Restore stashed work	Coming back to your task

The Git Workflow (Visual)



Common File Statuses in `git status`

Status	Meaning	Next Step
Untracked	Git sees it but isn't tracking	<code>git add</code>
Modified	File changed, not staged	<code>git add</code>
Staged	Ready to commit	<code>git commit</code>
Clean/Committed	Everything matches	Nothing!

Trouble? Undo!

Situation	Command
Staged wrong file	<code>git restore --staged filename</code>
Made bad edit, not staged	<code>git restore filename</code>
Bad commit, not pushed	<code>git reset --soft HEAD~1</code>
Bad commit, already pushed	<code>git revert HEAD</code>
Forgot what you did	<code>git log --oneline</code> or <code>git diff</code>

Appendix B: Instructor Timing Cheatsheet

Session	Slides	Lab	Exercise	Review	Total
1: First Repo	15 min	30 min	10 min	5 min	60 min
2: Bug Fix	15 min	30 min	10 min	5 min	60 min
3: GitHub PR	15 min	30 min	10 min	5 min	60 min
4: Revert + Conflict	15 min	30 min	10 min	5 min	60 min
5: CI/CD Actions	15 min	30 min	10 min	5 min	60 min

Ways to Shorten if Running Late

- Skip the `git stash` bonus in Session 4 (saves 5 min)

- Skip branch protection demo in Session 5 (saves 5 min)
- Do Session 3's PR exercise as a demo instead of hands-on (saves 10 min)
- Combine Session 1 and 2 if students are very technical (advanced groups only)

Appendix C: Glossary (From 5-Year-Old to Professional)

Simple Term	Professional Term	What It Means
Folder	Repository	A project that Git watches
Photo	Commit	A saved snapshot of files
Prep box	Staging Area	Files waiting to be committed
Time machine	Version Control	System that saves and restores history
Library	Remote / Origin	Server copy of the repo (GitHub)
Closed lane	Branch	A parallel line of development
Combine lanes	Merge	Joining two branches
Permission form	Pull Request	Asking to merge code into main
Robot	CI/CD Workflow	Automatic checks on code
Recipe	YAML file	Instructions for the robot

End of Approach 3: Story-Mode Curriculum